

“AI-Powered Humanoid Robot Fleet Management System”

Team: Robo-Nexus

Team Members:

Kiran Khatri	kkhatri@sfsu.edu
Kerry Yu	kyu19@sfsu.edu
Dias Almat	dalmat@sfsu.edu
Ishaank Zalpuri	izalpuri@sfsu.edu
Etienne Ghashehbaba	eghashehbaba@mail.sfsu.edu

GitHub Repository:

<https://github.com/sfsu-joseo/csc675-775-database-api-project-robo-nexus>

Version	Description
V1	initial submission for review

Table of contents

1. [Project Description](#)
2. [Functional Database Requirements](#)
3. [Non-Functional Database Requirements](#)

Project Description

This project focuses on designing a database system to manage fleets of humanoid robots used in environments such as warehouses, hospitals, and homes. As humanoid robots become more common, they generate a large amount of information, including task assignments, task execution history, deployment locations, energy levels, maintenance records, and interaction details. Currently, much of this information is either spread across different tools or not stored in a structured way, making it difficult to track robot behavior and performance over time.

There are existing robot fleet management platforms, such as `Formant` and `InOrbit`, that help monitor robots and show their current status. These systems are useful for real-time monitoring and coordination, but they are mainly designed for general robot fleets and do not focus on organizing long-term data for humanoid robots. In particular, they do not emphasize storing detailed interaction context, coordination roles, or historical data in a centralized database.

One unique feature of the system is its ability to store *human emotional context data* in personal and healthcare environments. In settings such as hospitals or homes, humanoid robots often interact closely with people. The system allows simple emotional observations, such as calm or urgent situations, to be recorded as data for later analysis. Most existing fleet management tools do not capture or organize this type of information.

Another unique feature is *role-based robot coordination* in warehouse environments. The system allows certain robots to be assigned a *senior* role to help coordinate tasks among other robots based on factors such as hardware capability and energy level. If a senior robot becomes unavailable, the system supports reassigning that role to another robot, helping prevent disruptions. Existing tools typically treat all robots equally and do not support this type of structured coordination with a fail-safe mechanism.

Overall, this project provides a centralized and structured way to manage humanoid robot data that existing solutions do not fully address. Manufacturers, warehouses, hospitals, and households deploying humanoid robots can benefit from a system that keeps robot information organized, reliable, and easy to understand over time.

Functional Database Requirements

1. Robot (Strong)

- 1.1. A robot shall belong at most one robot model.
- 1.2. A robot shall be deployed at at most one deployment location at a time.
- 1.3. A robot shall be assigned at most one role at a time.
- 1.4. A robot shall have many task assignments.
- 1.5. A robot shall participate in many task executions.
- 1.6. A robot shall have many emotion records over time.
- 1.7. A robot shall have at most one user.
- 1.8. A robot shall have many energy status records over time.
- 1.9. A robot shall generate many logs.
- 1.10. A robot may be associated with many RobotHardware.
- 1.11. A robot shall have at most one software.
- 1.12. A robot shall have many performance metrics.
- 1.13. A robot shall generate many alerts.
- 1.14. A robot shall have many diagnostic reports over time.
- 1.15. A robot shall have many sensors
- 1.16. A robot shall charge at many stations
- 1.17. A robot shall have many charging sessions
- 1.18. A robot shall have a current status

2. Robot Model (Strong)

- 2.1. A robot model shall support zero or many hardware components.
- 2.2. A robot model shall be produced by at most one manufacturer.
- 2.3. A robot model shall be associated with many robots.

3. Operator (User) (Strong)

- 3.1. An operator shall interact with many robots.
- 3.2. An operator may participate in many task assignments.
- 3.3. An operator may generate many support requests.
- 3.4. An operator may participate in many interaction sessions.
- 3.5. An operator shall be assigned many deployment locations.

4. **Manufacturer (Strong)**

- 4.1. A manufacturer shall produce many robot models.
- 4.2. A manufacturer shall produce many hardware.
- 4.3. A manufacturer shall provide zero or many software versions.

5. **Hardware(Strong)**

- 5.1. A hardware component shall be produced by exactly one manufacturer.
- 5.2. A hardware component shall participate in many robot hardware records.(loop)
- 5.3. A hardware component shall be supported by many robot models.(loop)

6. **Robot Hardware(Weak)**

- 6.1. A robot hardware record shall belong to exactly one robot.
- 6.2. A robot hardware record shall belong to exactly one hardware component.

7. **Software Version (Strong)**

- 7.1. A software version shall be associated with many robots
- 7.2. A software version shall be provided by exactly one manufacturer.

8. **Deployment Location (Strong-Superclass)**

- 8.1. A deployment location shall host many robots.
- 8.2. A deployment location shall support different environment types (warehouse, hospital, household).
- 8.3. A deployment location shall belong to at most one address.
- 8.4. A deployment location shall host many users
- 8.5. A deployment location shall have many logs
- 8.6. A deployment location shall have many charging sessions

9. **Address (Strong)**

- 9.1. An address may be associated with many deployment locations.
- 9.2. An address shall store street, city, state, and zipcode information.

10. Task (Strong)

- 10.1. A task shall be associated with many task assignments.
- 10.2. A task shall be associated with many task executions.

11. Task Assignment (weak)

- 11.1. A task assignment shall associate exactly one robot .
- 11.2. A task assignment shall belong to exactly one operator.
- 11.3. A task assignment shall belong to exactly one task.

12. Task Execution (Weak)

- 12.1. A task execution shall belong to exactly one robot.
- 12.2. A task execution shall belong to exactly one task.
- 12.3. A task execution shall generate at least one log

13. Emotion Record (Weak)

- 13.1. An emotion record shall belong to exactly one interaction session.
- 13.2. An emotion record shall store observed emotional state and timestamp.

14. InteractionSession(Weak)

- 14.1. An interaction session shall belong to exactly one robot.
- 14.2. An interaction session shall belong to exactly one operator.

15. Robot Role (Strong)

- 15.1. A robot role shall be assigned to many robots.
- 15.2. A robot role may be recommended by AI responses.

16. Energy Status (Weak)

- 16.1. An energy status shall be associated with exactly one robot.
- 16.2. An energy status shall be linked to one charging session.

17. PerformanceMetric(Weak)

- 17.1. A performance metrics shall belong to exactly one robot.
- 17.2. A performance shall have a time stamp record.

18. Alert(Weak)

- 18.1. An alert shall belong to exactly one robot.
- 18.2. An alert shall record severity and time.
- 18.3. An alert may be associated with one AI request.

19. DiagnosticReport(Weak)

- 19.1. A diagnostic report shall belong to exactly one robot.
- 19.2. A diagnostic report shall have performance metrics.
- 19.3. A diagnostic report shall be associated with one AI model.

20. Log (weak)

- 20.1. A log shall be associated with one robot.
- 20.2. A log shall record operational events.
- 20.3. A log shall be associated with at most one deployment location

21. Sensor (Weak)

- 21.1. A sensor shall be associated with one robot
- 21.2. A sensor shall generate at least one log

22. Charging Station (Strong)

- 22.1. A charging station shall serve many robots
- 22.2. A charging station shall record many charging sessions

23. Charging Session (Weak)

- 23.1. A charging session shall occur at one charging station
- 23.2. A charging session shall be associated with one robot
- 23.3. A charging session shall be associated with one deployment location
- 23.4. A charging session shall generate at least one log

24. Support Request (Weak)

- 24.1. A support request shall belong to exactly one robot.
- 24.2. A support request shall belong to exactly one operator.
- 24.3. A support request shall record issue details and time.

25. AI Model (Strong)

- 25.1. An AI model shall generate many AI requests.

- 25.2. An AI model shall analyze zero or many diagnostic reports.
- 25.3. An AI model shall store service type.

26. AI Request(Weak)

- 26.1. An AI request shall belong to exactly one AI model.
- 26.2. An AI request shall reference one robot.
- 26.3. An AI request shall analyze at least one emotion record.

27. AI Response (Weak)

- 27.1. An AI response shall belong to exactly one AI request.
- 27.2. An AI response shall recommend exactly one robot role.
- 27.3. An AI response shall recommend one task.

28. Hospital (Strong-Subclass)

- 28.1. A hospital shall belong to at most one deployment location
- 28.2. A hospital shall have many departments.
- 28.3. A hospital shall have many emotion records.

29. Warehouse (Strong-Subclass)

- 29.1. A warehouse shall belong to at most one deployment location
- 29.2. A warehouse shall host many robots.
- 29.3. A warehouse shall have many storage zones.

30. Home(Strong-Subclass)

- 30.1. A home shall belong to at most one deployment location
- 30.2. A home may host many robots.
- 30.3. A home may record many interaction sessions.

31. Department(Weak)

- 31.1. A department shall belong to exactly one hospital.

- 31.2. A department may host many robots.
- 31.3. A department may record many interaction sessions.

32. EmotionalPattern (Weak)

- 32.1. An emotional pattern shall belong to exactly one robot.
- 32.2. An emotional pattern shall summarize at least one emotion record.
- 32.3. An emotional pattern shall be analyzed by exactly one AI model.

33. AccessPermission (Strong)

- 33.1. An access permission shall belong to at most one robot role.
- 33.2. An access permission shall define access level.
- 33.3. An access permission shall apply to many operators.

34. UserRole (Strong)

- 34.1. A user role shall classify many operators.
- 34.2. A user role shall be associated with many access permissions.
- 34.3. A user role shall define authority level.

35. MaintenanceRecord (Weak)

- 35.1. A maintenance record shall belong to exactly one robot.
- 35.2. A maintenance record shall reference zero or many hardware components.
- 35.3. A maintenance record shall be performed by exactly one technician.

Non-Functional Database Requirements

1. Performance

- 1.1. The system shall support concurrent access by at least _ robots without performance degradation.
- 1.2. The system shall be able to handle concurrent usage by _ robots
- 1.3. The system shall be able to maintain performance while receiving AI driven queries for real time data

2. Security

- 2.1. Any logs and data shall only be able to be accessed by an user with sufficient permissions
- 2.2. The system shall protect AI request and AI response data from unauthorized access.

3. Reliability

- 3.1. The system shall retain operational logs for at least 12 months.
- 3.2. The system shall have daily backups

4. Maintainability

- 4.1. The system shall host AI queries and structural queries separately
- 4.2. The database schema shall be well documented.

5. Scalability

- 5.1. The system shall support growth in the number of robots and deployment locations.
- 5.2. The system shall support increasing volumes of task execution and logs data.
- 5.3. The system shall support any future AI integration.

6. Privacy

- 6.1. Emotional interaction records shall be stored securely to protect human-related sensitive data.
 - 6.2. Access to emotional data shall be restricted to authorized persons only .
- .